

ECE 759

High Performance Computing for Engineering Applications

Assignment 2

Submitted by: Zhengyang Wan

My GitHub link: <https://github.com/BillWan-zzzyyy/repo759/tree/main/HW02>

1.

- (a) Implement the `scan` function in a file called `scan.cpp` with signature defined as in `scan.h`. You should write an inclusive scan on your own and not use any library scan functions.

Answer: Submitted as HW02/scan.cpp in the GitHub repository:

```
#include "scan.h"

void scan(const float *arr, float *output, std::size_t n) {
    // nothing to do for an empty array (also avoids reading arr[0])
    if (n == 0) {
        return;
    }

    // inclusive scan: output[i] = arr[0] + arr[1] + ... + arr[i]
    // each step reuses the previous partial sum, so this is O(n)
    output[0] = arr[0];
    for (std::size_t i = 1; i < n; ++i) {
        output[i] = output[i - 1] + arr[i];
    }
}
```

- (b) Write a file `task1.cpp` with a `main` function which (in this order)
- Creates an array of `n` random `float` numbers between -1.0 and 1.0. `n` should be read as the first command line argument as below.
 - Scans the array using your `scan` function.
 - Prints out the time taken by your `scan` function in *milliseconds*.
 - Prints the first element of the output scanned array.
 - Prints the last element of the output scanned array.

vi) Deallocates memory when necessary.

Compile command: `g++ scan.cpp task1.cpp -Wall -O3 -std=c++17 -o task1`

Run command: `./task1 n`

Answer: Submitted as HW02/task1.cpp in the GitHub repository:

```
#include <cerrno>
#include <chrono>
#include <cstdlib>
#include <iostream>
#include <random>

#include "scan.h"

using std::chrono::duration;
using std::chrono::high_resolution_clock;

int main(int argc, char* argv[]) {
    if (argc != 2) {
        std::cerr << "Usage: " << argv[0] << " n\n";
        return 1;
    }

    // read n and make sure it is a positive integer
    char* end = nullptr;
    errno = 0;
    const long parsed = std::strtoul(argv[1], &end, 10);
    if (errno != 0 || end == argv[1] || *end != '\0' || parsed <= 0) {
        std::cerr << "n must be a positive integer.\n";
        return 1;
    }
    const std::size_t n = static_cast<std::size_t>(parsed);

    float* arr = new float[n];
    float* output = new float[n];

    // fill the input with random floats in [-1, 1]
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_real_distribution<float> dist(-1.0f, 1.0f);
    for (std::size_t i = 0; i < n; ++i) {
        arr[i] = dist(gen);
    }

    // only the scan call is timed, not the random number generation
    high_resolution_clock::time_point start = high_resolution_clock::now();
    scan(arr, output, n);
    high_resolution_clock::time_point stop = high_resolution_clock::now();
    duration<double, std::milli> elapsed = stop - start;

    // time in ms, then the first and last element of the scanned array
```

```

std::cout << elapsed.count() << "\n";
std::cout << output[0] << "\n";
std::cout << output[n - 1] << "\n";

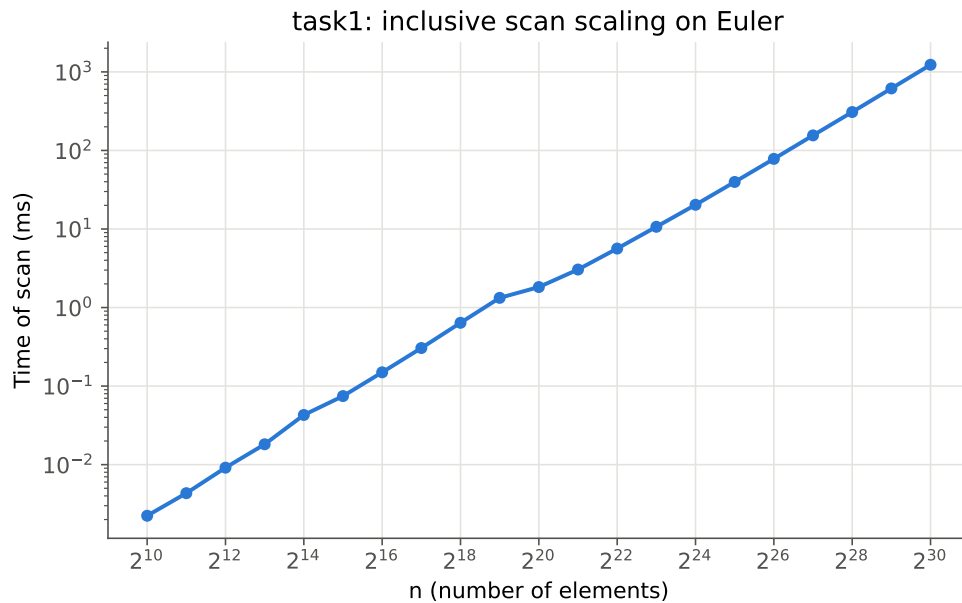
// free the arrays allocated with new[]
delete[] arr;
delete[] output;

return 0;
}

```

- (c) On an *Euler* compute node (using a Slurm Job), run `task1` for each value $n = 2^{10}, 2^{11}, \dots, 2^{30}$ and generate a plot `task1.pdf` (with axis labels) which plots the time taken by your algorithm as a function of n . This is called a scaling analysis.

Answer: The plot is submitted separately as `task1.pdf` and is also shown below. Both axes are in log scale. The time grows linearly with n : from about 2^{20} on, doubling n doubles the time, and at $n = 2^{30}$ the scan takes 1232 ms, i.e. about 1.15 ns per element. This matches the $O(n)$ work of a sequential inclusive scan, where each addition has to wait for the previous one.



2.

Convolutions appear very prominently in image processing and in other fields like the numerical solution of partial differential equations, machine learning, etc.

- (a) Implement the `convolve` function in a file called `convolution.cpp` with signature as provided in `convolution.h`. The operation that needs to be implemented is as follows:

$$g[x, y] = \sum_{i=0}^{m-1} \sum_{j=0}^{m-1} \omega[i, j] f \left[x + i - \frac{m-1}{2}, y + j - \frac{m-1}{2} \right],$$

where $x, y = 0, \dots, n-1$; f is the original image, ω is the mask, and g is the result of the convolution; m is the dimension of the square matrix ω , where $m \geq 1$ is assumed to be an odd number. We assume that $f[i, j] = 1$ if one of the following two conditions is not satisfied, and $f[i, j] = 0$ if neither of the following two conditions is satisfied:

$$\begin{cases} 0 \leq i < n, \\ 0 \leq j < n. \end{cases}$$

In other words, we will pad zeros for corners, and pad ones for edges (excluding the corners).

Example (uses $n = 4$ and $m = 3$):

$$f = \begin{bmatrix} 1 & 3 & 4 & 8 \\ 6 & 5 & 2 & 4 \\ 3 & 4 & 6 & 8 \\ 1 & 4 & 5 & 2 \end{bmatrix}, \quad \omega = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}, \quad g = \begin{bmatrix} 3 & 10 & 10 & 10 \\ 10 & 12 & 14 & 11 \\ 9 & 7 & 14 & 14 \\ 5 & 11 & 14 & 4 \end{bmatrix}$$

Answer: Submitted as HW02/convolution.cpp in the GitHub repository:

```
#include "convolution.h"

// Value of f[i, j] with the padding rule from HW02:
// inside the image -> image value, edges -> 1, corners -> 0.
static float get_pixel(const float *image, std::size_t n, long i, long j) {
    // use signed indices here since i or j can be negative near the border
    const long size = static_cast<long>(n);
    const bool i_in = (i >= 0 && i < size);
    const bool j_in = (j >= 0 && j < size);

    if (i_in && j_in) {
        return image[i * size + j];
    }
    // exactly one index is out of range -> edge
    if (i_in || j_in) {
        return 1.0f;
    }
    // both indices are out of range -> corner
    return 0.0f;
}

void convolve(const float *image, float *output, std::size_t n,
```

```

        const float *mask, std::size_t m) {
// (m - 1) / 2 shifts the mask so that its center sits on (x, y)
const long half = static_cast<long>((m - 1) / 2);

// loop over every pixel of the output image
for (std::size_t x = 0; x < n; ++x) {
    for (std::size_t y = 0; y < n; ++y) {
        float sum = 0.0f;
        // apply the m x m mask around (x, y)
        for (std::size_t i = 0; i < m; ++i) {
            for (std::size_t j = 0; j < m; ++j) {
                // row/col can fall outside the image, get_pixel handles padding
                const long row = static_cast<long>(x + i) - half;
                const long col = static_cast<long>(y + j) - half;
                sum += mask[i * m + j] * get_pixel(image, n, row, col);
            }
        }
        output[x * n + y] = sum;
    }
}
}

```

(b) Write a file `task2.cpp` with a `main` function which (in this order)

- i) Creates an $n \times n$ image matrix (stored in 1D in row-major order) of random float numbers between -10.0 and 10.0. The value of `n` should be read as the first command line argument.
- ii) Creates an $m \times m$ mask matrix (stored in 1D in row-major order) of random float numbers between -1.0 and 1.0. The value of `m` should be read as the second command line argument.
- iii) Applies the `mask` to `image` using your `convolve` function.
- iv) Prints out the time taken by your `convolve` function in *milliseconds*.
- v) Prints the first element of the resulting convolved array.
- vi) Prints the last element of the resulting convolved array.
- vii) Deallocates memory when necessary via the `delete` function.

Compile command: `g++ convolution.cpp task2.cpp -Wall -O3 -std=c++17 -o task2`

Run command: `./task2 n m`

Answer: Submitted as `HW02/task2.cpp` in the GitHub repository:

```

#include <cerrno>
#include <chrono>

```

```

#include <cstdlib>
#include <iostream>
#include <random>

#include "convolution.h"

using std::chrono::duration;
using std::chrono::high_resolution_clock;

// Returns the parsed value, or 0 if str is not a positive integer.
long parse_positive(const char* str) {
    char* end = nullptr;
    errno = 0;
    const long parsed = std::strtoul(str, &end, 10);
    if (errno != 0 || end == str || *end != '\0' || parsed <= 0) {
        return 0;
    }
    return parsed;
}

int main(int argc, char* argv[]) {
    if (argc != 3) {
        std::cerr << "Usage: " << argv[0] << " n m\n";
        return 1;
    }

    // n is the image size, m is the mask size (m must be odd to have a center)
    const long n_arg = parse_positive(argv[1]);
    const long m_arg = parse_positive(argv[2]);
    if (n_arg == 0 || m_arg == 0 || m_arg % 2 == 0) {
        std::cerr << "n must be a positive integer and m a positive odd integer.\n";
        return 1;
    }

    const std::size_t n = static_cast<std::size_t>(n_arg);
    const std::size_t m = static_cast<std::size_t>(m_arg);

    // both matrices are stored in 1D in row-major order
    float* image = new float[n * n];
    float* mask = new float[m * m];
    float* output = new float[n * n];

    // image values in [-10, 10], mask values in [-1, 1]
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_real_distribution<float> image_dist(-10.0f, 10.0f);
    std::uniform_real_distribution<float> mask_dist(-1.0f, 1.0f);

    for (std::size_t i = 0; i < n * n; ++i) {
        image[i] = image_dist(gen);
    }
    for (std::size_t i = 0; i < m * m; ++i) {

```

```

    mask[i] = mask_dist(gen);
}

// only the convolution itself is timed
high_resolution_clock::time_point start = high_resolution_clock::now();
convolve(image, output, n, mask, m);
high_resolution_clock::time_point stop = high_resolution_clock::now();
duration<double, std::milli> elapsed = stop - start;

// time in ms, then the first and last element of the result
std::cout << elapsed.count() << "\n";
std::cout << output[0] << "\n";
std::cout << output[n * n - 1] << "\n";

// free everything with delete[] as the task asks
delete[] image;
delete[] mask;
delete[] output;

return 0;
}

```

3.

Implement in a file called `matmul.cpp` the four functions with signatures and descriptions as in `matmul.h` to produce the matrix product $C = AB$. For all of the cases, the array `C` that stores the matrix C should be reported in row-major order.

- (a) `mmul1` should have three `for` loops: the outer loop sweeps index `i` through the rows of `C`, the middle loop sweeps index `j` through the columns of `C`, and the innermost loop sweeps index `k` through; i.e., to carry out, the dot product of the i^{th} row `A` with the j^{th} column of `B`. Inside the innermost loop, you should have a single line of code which increments C_{ij} . Assume that `A` and `B` are 1D arrays storing the matrices in row-major order.

Answer: Submitted in `HW02/matmul.cpp` in the GitHub repository (all four functions live in this file; `C` is zeroed first because each version only accumulates into it):

```

// mmul1: loop order (i, j, k). The inner loop is the dot product of row i
// of A and column j of B, so B is read with a stride of n.
void mmul1(const double* A, const double* B, double* C, const unsigned int n) {
    std::fill(C, C + n * n, 0.0);
    for (unsigned int i = 0; i < n; ++i) {
        for (unsigned int j = 0; j < n; ++j) {
            for (unsigned int k = 0; k < n; ++k) {
                C[i * n + j] += A[i * n + k] * B[k * n + j];
            }
        }
    }
}

```

```

    }
  }
}

```

- (b) `mmul2` should also have three `for` loops, but the two innermost loops should be swapped relative to `mmul1` ((i, j, k) becomes (i, k, j)).

Answer:

```

// mmul2: loop order (i, k, j). A[i][k] is fixed in the inner loop and
// row k of B and row i of C are walked through contiguously.
void mmul2(const double* A, const double* B, double* C, const unsigned int n) {
    std::fill(C, C + n * n, 0.0);
    for (unsigned int i = 0; i < n; ++i) {
        for (unsigned int k = 0; k < n; ++k) {
            for (unsigned int j = 0; j < n; ++j) {
                C[i * n + j] += A[i * n + k] * B[k * n + j];
            }
        }
    }
}

```

- (c) `mmul3` should also have three `for` loops, but the outermost loop in `mmul1` should become the innermost loop in `mmul3` ((i, j, k) becomes (j, k, i)).

Answer:

```

// mmul3: loop order (j, k, i). B[k][j] is fixed in the inner loop, but
// A and C are both walked down a column (stride n).
void mmul3(const double* A, const double* B, double* C, const unsigned int n) {
    std::fill(C, C + n * n, 0.0);
    for (unsigned int j = 0; j < n; ++j) {
        for (unsigned int k = 0; k < n; ++k) {
            for (unsigned int i = 0; i < n; ++i) {
                C[i * n + j] += A[i * n + k] * B[k * n + j];
            }
        }
    }
}

```

- (d) `mmul4` should have the `for` loops ordered as in `mmul1`, but this time around `A` and `B` are stored as `std::vector<double>`.

Answer:

```

// mmul4: same loops as mmul1, only A and B are std::vector<double>.
void mmul4(const std::vector<double>& A, const std::vector<double>& B,
           double* C, const unsigned int n) {
    std::fill(C, C + n * n, 0.0);
    for (unsigned int i = 0; i < n; ++i) {

```

```

    for (unsigned int j = 0; j < n; ++j) {
        for (unsigned int k = 0; k < n; ++k) {
            C[i * n + j] += A[i * n + k] * B[k * n + j];
        }
    }
}
}
}

```

- (e) Write a program `task3.cpp` that generates square matrices `A` and `B` of dimension at least 1000×1000 stored in row-major order, computes $C = AB$ using each of your functions, and prints the number of rows of your input matrices, and for each `mmul` function in ascending order, prints the amount of time taken in *milliseconds* and the last element of the resulting `C` (nine values, one per line).

Compile command: `g++ task3.cpp matmul.cpp -Wall -O3 -std=c++17 -o task3`

Run command: `./task3`

Answer: Submitted as `HW02/task3.cpp` in the GitHub repository:

```

#include <chrono>
#include <iostream>
#include <random>
#include <vector>

#include "matmul.h"

using std::chrono::duration;
using std::chrono::high_resolution_clock;

int main() {
    // the task asks for at least 1000 x 1000, so use 1024
    const unsigned int n = 1024;

    // mmul1-3 take raw arrays, mmul4 takes vectors, so keep both copies.
    double* A = new double[n * n];
    double* B = new double[n * n];
    double* C = new double[n * n];

    // random entries in [-1, 1], stored in row-major order
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_real_distribution<double> dist(-1.0, 1.0);
    for (unsigned int i = 0; i < n * n; ++i) {
        A[i] = dist(gen);
        B[i] = dist(gen);
    }
    // same values as A and B, so all four versions compute the same C
    std::vector<double> A_vec(A, A + n * n);
    std::vector<double> B_vec(B, B + n * n);

    high_resolution_clock::time_point start;

```

```

high_resolution_clock::time_point stop;
duration<double, std::milli> elapsed;

std::cout << n << "\n";

// for each version: time in ms, then the last element of C
start = high_resolution_clock::now();
mmul1(A, B, C, n);
stop = high_resolution_clock::now();
elapsed = stop - start;
std::cout << elapsed.count() << "\n";
std::cout << C[n * n - 1] << "\n";

start = high_resolution_clock::now();
mmul2(A, B, C, n);
stop = high_resolution_clock::now();
elapsed = stop - start;
std::cout << elapsed.count() << "\n";
std::cout << C[n * n - 1] << "\n";

start = high_resolution_clock::now();
mmul3(A, B, C, n);
stop = high_resolution_clock::now();
elapsed = stop - start;
std::cout << elapsed.count() << "\n";
std::cout << C[n * n - 1] << "\n";

start = high_resolution_clock::now();
mmul4(A_vec, B_vec, C, n);
stop = high_resolution_clock::now();
elapsed = stop - start;
std::cout << elapsed.count() << "\n";
std::cout << C[n * n - 1] << "\n";

delete[] A;
delete[] B;
delete[] C;

return 0;
}

```

Output on an Euler compute node:

```

1024
6645.41
1.97072
626.513
1.97072
14872.1
1.97072
7377.4

```

- (f) In a couple sentences, explain the difference that you see in the times for `mmul1`, `mmul2` and `mmul3` when running on an Euler compute node. What would explain the performance results you report? Be as specific as possible. Also comment on the difference or similarity you see between `mmul1` and `mmul4`.

Answer: On Euler with $n = 1024$ I got `mmul1` 6645 ms, `mmul2` 627 ms, `mmul3` 14872 ms and `mmul4` 7377 ms, and all four give the same last element (1.97072). So `mmul2` is about $10\times$ faster than `mmul1`, and `mmul3` is about $2\times$ slower than `mmul1`.

The difference comes from the memory access pattern of the innermost loop, since all matrices are stored in row-major order. In `mmul2` (`i,k,j`) the inner loop walks along row `k` of `B` and row `i` of `C` with stride 1 while `A[i*n+k]` stays fixed, so every 64-byte cache line that is loaded is fully used (8 doubles) and the compiler can vectorize the loop. In `mmul1` (`i,j,k`) `A` is still read along a row, but `B` is read down a column, jumping $n \cdot 8$ bytes = 8 KB every iteration, so almost every access to `B` is a cache miss. In `mmul3` (`j,k,i`) both `A[i*n+k]` and `C[i*n+j]` are accessed down a column with the 8 KB stride, and `C` is also written back, so there are two strided streams instead of one and it is the slowest. Since $n = 1024$ is a power of two, the elements of one column also map to the same few cache sets and keep evicting each other, which makes the column-wise versions even worse.

`mmul4` uses the same loop order as `mmul1`, and its time is about the same (around 10% apart). With `-O3`, `std::vector`'s `operator[]` is inlined into a plain pointer access, so storing `A` and `B` in a vector does not change the access pattern; the small gap is most likely just run-to-run noise.